Uma Heurística Gulosa por Classes de Cor para o Problema de Coloração Seletiva de Grafos

Carlos Alberto de Lima Neto, Guilherme Augusto Pondofe Santana

[1]Departamento de Informática e Matemática Aplicada (DIMAp)
Universidade Federal do Rio Grande do Norte (UFRN) – Natal – RN – Brasil

Resumo. *No Problema de Coloração Seletiva de Grafos, os vértices de um grafo estão divididos em k grupos, e é preciso escolher exatamente um vértice de cada grupo e colorir os escolhidos com o menor número de cores. O problema generaliza a coloração de vértices e, por isso, é NP-difícil e difícil de aproximar. Implementamos uma heurística construtiva gulosa que monta uma classe de cor por vez, provamos que ela roda em tempo $O(n \log n + cn + m)$ e usa no máximo $1 + \max_i \min_{v \in V_i} d_i(v)$ cores, e a avaliamos em 280 instâncias da literatura. Todas as soluções foram válidas, a maior instância levou 3,4 ms, e o gap médio em relação aos valores ótimos ou melhores conhecidos foi de 19% nas instâncias nsfnet e ring e de 43% nas instâncias random.*

1. Introdução

Problemas de coloração de grafos modelam situações em que recursos precisam ser atribuídos a elementos que entram em conflito. Na coloração seletiva, além de colorir, é preciso escolher *quais* elementos serão coloridos: os vértices estão organizados em grupos e apenas um vértice de cada grupo entra na solução. O problema foi proposto por [Li and Simha 2000], com o nome de *Partition Coloring Problem* (PCP), para modelar o roteamento e a atribuição de comprimentos de onda em redes ópticas: cada grupo reúne as rotas possíveis de uma conexão, duas rotas que compartilham um enlace são adjacentes e cada cor é um comprimento de onda. Outras aplicações, como problemas de escalonamento e de alocação, são discutidas por [Demange et al. 2015].

Uma forma de ver o problema é a seguinte. Há tarefas divididas em categorias, e de cada categoria basta executar uma tarefa. Duas tarefas são incompatíveis quando acontecem no mesmo horário. Cada trabalhador executa um conjunto de tarefas compatíveis entre si. Queremos escolher uma tarefa de cada categoria e contratar o menor número de trabalhadores capaz de executá-las.

Este trabalho apresenta uma heurística construtiva para o problema. As contribuições são: (i) a implementação de uma heurística gulosa que monta uma classe de cor por vez, inspirada na heurística inicial de [Furini et al. 2018], e de um método de duas fases usado como referência de comparação; (ii) a análise de complexidade de tempo da heurística e um limite superior para o número de cores que ela usa; e (iii) a aplicação dos dois métodos às 280 instâncias clássicas da literatura, com comparação aos valores ótimos publicados e a heurísticas construtivas da literatura.

O restante do texto está organizado assim. A Seção 2 define o problema e a Seção 3 discute a sua complexidade. A Seção 4 resume os métodos já publicados e a Seção 5 apresenta os bancos de instâncias e links úteis. A Seção 6 descreve o algoritmo e a sua implementação, analisados na Seção 7. A Seção 8 traz os experimentos e a Seção 9 conclui o trabalho.

2. Descrição do problema

Seja $G = (V, E)$ um grafo simples com $n = |V|$ vértices e $m = |E|$ arestas, e seja $\mathcal{P} = \{V_1, \ldots, V_k\}$ uma partição de V em k grupos não vazios. Uma *seleção* é um conjunto $S \subseteq V$ com $|S \cap V_i| = 1$ para todo i, isto é, um representante por grupo. Uma *coloração própria* de S é uma função $f : S \to \{1, \ldots, c\}$ tal que $f(u) \neq f(v)$ para toda aresta $uv \in E$ com $u, v \in S$. O problema consiste em encontrar uma seleção S e uma coloração própria de $G[S]$ com o menor número possível de cores. Esse mínimo é o *número cromático de partição* de G, denotado por $\chi_P(G)$:

$$\chi_P(G) = \min\{\chi(G[S]) : S \text{ é uma seleção}\},$$

onde χ é o número cromático usual. Arestas entre vértices de um mesmo grupo nunca aparecem em $G[S]$ e podem ser ignoradas. Para um vértice $v \in V_i$, escrevemos $d_i(v) = |N(v) \setminus V_i|$ para o número de vizinhos de v fora do seu grupo.

A Figura 1 mostra a instância de exemplo usada nos testes do projeto. Os vértices ímpares formam um K_4, de modo que escolher o primeiro vértice de cada grupo exige quatro cores. Escolhendo os vértices $\{2, 3, 6, 8\}$, duas cores bastam, e esse é o ótimo, pois nenhuma seleção é um conjunto independente.

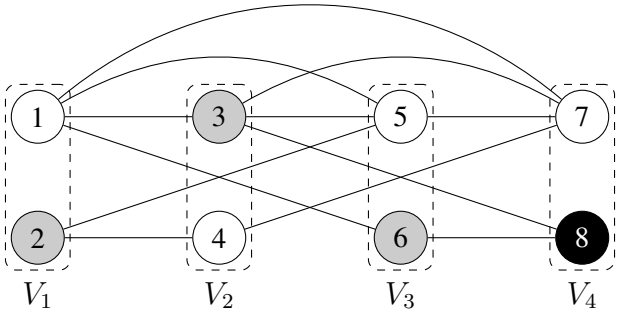


Figura 1. Instância de exemplo com $n = 8$, $m = 12$ e $k = 4$. Os vértices preenchidos formam a solução ótima encontrada pela heurística: 2, 3 e 6 com a cor 1 (cinza) e 8 com a cor 2 (preto).

3. Complexidade do problema

A versão de decisão do problema ("existe seleção colorível com no máximo c cores?") pertence a NP: uma seleção e uma coloração formam um certificado verificável em tempo $O(n + m)$. O problema é NP-difícil porque contém a coloração de vértices como caso particular: quando todo grupo tem um único vértice, a única seleção é $S = V$ e $\chi_P(G) = \chi(G)$. Como decidir se $\chi(G) \leq 3$ é NP-completo [Karp 1972, Garey and Johnson 1979], decidir se $\chi_P(G) \leq 3$ também é [Li and Simha 2000]. A coloração seletiva continua NP-difícil mesmo em classes de grafos nas quais a coloração usual é fácil, conforme mostrado por [Demange et al. 2014].

A mesma redução vale para a aproximação. [Zuckerman 2007] mostrou que, para todo $\varepsilon > 0$, não existe algoritmo polinomial que aproxime $\chi(G)$ dentro de um fator $n^{1-\varepsilon}$, a menos que P = NP. Na redução acima, $k = n$, de modo que o PCP também não admite aproximação dentro de $k^{1-\varepsilon}$ sob a mesma hipótese. Em particular, não se espera algoritmo aproximativo com razão constante para o problema, o que justifica o uso de heurísticas.

4. Heurísticas e algoritmos da literatura

Heurísticas construtivas. [Li and Simha 2000] adaptaram ao PCP três heurísticas clássicas de coloração: *Largest-First* [Welsh and Powell 1967], *Smallest-Last* e *Color-Degree*, esta última próxima do DSATUR [Brélaz 1979]. Cada uma tem uma versão de duas fases (*Twostep*: primeiro escolhe os representantes, depois colore) e uma de uma fase (*Onestep*: escolhe e colore ao mesmo tempo). A variante Onestep-CD, que a cada passo colore o vértice de maior grau de cor entre os grupos ainda não atendidos, foi a melhor delas e serviu de solução inicial para vários trabalhos posteriores. [Fritz 2013] propôs uma construção híbrida que combina Onestep-CD com a heurística DANGER.

Metaheurísticas. [Noronha and Ribeiro 2006] partem da solução de Onestep-CD e reduzem o número de cores com busca tabu, superando as heurísticas construtivas. [Pop et al. 2013] propuseram um algoritmo memético que combina operadores genéticos e busca local. [Furini et al. 2018] propuseram uma busca adaptativa em vizinhança grande (ALNS) com busca local, cuja solução inicial é gerada por uma heurística gulosa que constrói um conjunto independente por vez, a mesma ideia do método implementado neste trabalho.

Algoritmos exatos. [Frota et al. 2010] apresentaram um algoritmo branch-and-cut baseado na formulação por representantes. [Hoshino et al. 2011] propuseram um branch-and-price sobre uma reformulação de Dantzig-Wolfe da mesma formulação. [Furini et al. 2018] propuseram um novo branch-and-price, com subproblema de conjunto independente de peso máximo, que resolveu pela primeira vez várias instâncias em aberto. [Şeker et al. 2021] propuseram um algoritmo exato de planos de corte para o caso em que o grafo é perfeito, junto com um gerador de instâncias desse tipo. Os valores ótimos publicados nesse trabalho são a principal referência dos nossos experimentos.

Algoritmos aproximativos. Pelo resultado de inaproximabilidade da Seção 3, não se conhecem algoritmos aproximativos com razão constante para o PCP, e não se espera que existam. Os métodos práticos são heurísticos ou exatos, como os citados acima.

5. Bancos de instâncias e links

Usamos o banco clássico de instâncias do PCP, distribuído pelo grupo de C. C. Ribeiro e mantido no repositório de [Santini 2017], que acompanha [Furini et al. 2018]. A Tabela 1 resume os três conjuntos.

- **random** [Frota et al. 2010]: grafos aleatórios com grupos de dois vértices. Há um conjunto com densidade 0,5 e n de 20 a 120, e outro com $n = 90$ e densidade de 0,1 a 0,9, com cinco instâncias por combinação.
- **nsfnet** [Li and Simha 2000]: roteamento na rede NSF (14 nós e 21 enlaces). Cada conexão pedida com probabilidade $p \in \{0{,}1, \ldots, 1{,}0\}$ gera um grupo com até duas rotas, e duas rotas são adjacentes quando compartilham um enlace.
- **ring** [Frota et al. 2010]: o mesmo processo em redes em anel com 10, 15 e 20 nós. Os cinco arquivos `ring_n{10,15,20}p1.0s{1..5}` de cada tamanho são cópias idênticas, como notam [Furini et al. 2018]; eles foram mantidos para preservar os nomes originais.

Tabela 1. Conjuntos de instâncias usados nos experimentos.

Conjunto	Arquivos	n	k	Densidade	Origem
random	80	20–120	10–60	0,10–0,91	[Frota et al. 2010]
nsfnet	50	16–257	12–182	0,09–0,13	[Li and Simha 2000]
ring	150	8–760	4–380	0,36–0,41	[Frota et al. 2010]

As instâncias foram convertidas para um formato textual simples, com vértices e grupos numerados a partir de 1: uma linha `p edge` n m k, uma linha `c` i v_1 v_2 `...` por grupo e uma linha `e` u v por aresta. Como existem instâncias da literatura com tamanho significativo (até 760 vértices e 117 mil arestas), não foi necessário criar um gerador, como o de grafos perfeitos de [Şeker et al. 2021]. Usar as instâncias clássicas também permite comparar os resultados com os valores ótimos publicados.

Links importantes:

- Repositório deste trabalho (código, instâncias e resultados): `https://github.com/pondof/trab1PAA`.
- Instâncias e código de [Furini et al. 2018]: `https://github.com/alberto-santini/selective-graph-colouring`.
- Página do grupo de C. C. Ribeiro sobre o PCP: `http://www2.ic.uff.br/~celso/grupo/pcp.htm`.

6. Algoritmo desenvolvido

6.1. Guloso por classes de cor

A heurística principal, chamada `guloso_classes`, contrata um trabalhador (cor) por vez e dá a ele o maior número possível de tarefas compatíveis, no máximo uma por grupo. Os vértices são percorridos em ordem crescente de grau, com desempate pelo menor identificador: vértices de grau baixo conflitam com poucos outros e tendem a deixar espaço para mais vértices na mesma cor. Um vértice entra na cor atual se o seu grupo ainda não foi atendido e se ele não é vizinho de nenhum vértice já colocado nessa cor. Quando a varredura termina, a cor está fechada e uma nova cor é aberta. O processo para quando todos os grupos têm representante. O Algoritmo 1 resume o método, baseado na heurística gulosa do código de [Santini 2017].

Na instância da Figura 1, a ordem é $2, 4, 6, 8, 1, 3, 5, 7$ (os pares têm grau 2 e os ímpares, grau 4). Na cor 1 entram 2, 6 e 3; os vértices 4, 8 e 7 ficam bloqueados por vizinhos já escolhidos. Na cor 2 entra o vértice 8, atendendo o último grupo. A solução usa duas cores, que é o ótimo.

6.2. Método de duas fases (referência)

Para ter uma referência de comparação, implementamos também o método `duas_fases`, uma versão Twostep simples: (1) em cada grupo, escolhe o vértice de menor grau, com desempate pelo menor identificador; (2) colore os escolhidos com *first-fit* na ordem dos grupos, dando a cada um a menor cor não usada pelos seus vizinhos já coloridos.

Algoritmo 1 Guloso por classes de cor

Entrada: grafo $G = (V, E)$ e partição $\{V_1, \ldots, V_k\}$
Saída: representante r_i e cor f_i de cada grupo V_i

$L \leftarrow$ vértices de V em ordem crescente de $(\deg(v), v)$
$P \leftarrow \{1, \ldots, k\}$ ▷ grupos pendentes
$c \leftarrow 0$
enquanto $P \neq \emptyset$ **faça**
    $c \leftarrow c + 1$; $B \leftarrow \emptyset$ ▷ B: vizinhos da cor c
    para cada $v \in L$ **faça**
        $i \leftarrow$ grupo de v
        se $i \in P$ **e** $v \notin B$ **então**
            $r_i \leftarrow v$; $f_i \leftarrow c$; $P \leftarrow P \setminus \{i\}$
            $B \leftarrow B \cup N(v)$
        fim se
    fim para
fim enquanto
devolva (r, f)

6.3. Implementação

Os métodos foram implementados em Python 3, usando apenas a biblioteca padrão. O grafo é guardado como uma lista de conjuntos de adjacência, o que torna $O(1)$ (esperado) o teste de adjacência. O código está organizado assim:

- `sgcp.py`: leitura das instâncias, os dois métodos e o validador;
- `main.py`: interface de terminal. Um menu permite escolher o conjunto e depois a instância; os dois métodos são executados e um relatório com os dados da instância, o número de cores, o tempo e a validação de cada método e a melhor solução (representante e cor de cada grupo e tarefas de cada trabalhador) é mostrado na tela e salvo em `resultados/`;
- `experimentos.py`: executa os dois métodos em todas as instâncias e gera as tabelas de resultados;
- `testes.py`: testes automatizados com a instância de exemplo.

Toda solução passa por um validador antes de ser mostrada ou salva: ele confere que cada grupo tem um representante que pertence ao grupo e que nenhuma aresta liga dois representantes da mesma cor. Para executar, basta, na pasta do projeto, rodar `python main.py` (menu), `python main.py instancias/exemplo/toy.txt` (instância direta) ou `python experimentos.py` (todas as instâncias). Em alguns sistemas o comando é `python3`.

7. Análise do algoritmo

7.1. Complexidade de tempo

Seja c o número de cores usado. A ordenação inicial custa $O(n \log n)$. Cada iteração do laço externo percorre os n vértices de L, com testes de pertinência $O(1)$, e há c iterações, totalizando $O(cn)$. A atualização $B \leftarrow B \cup N(v)$ é executada uma única vez por

grupo, quando o seu representante é escolhido, e custa $O(\deg(v))$; somando, o custo é no máximo $\sum_v \deg(v) = 2m$. O tempo total é, portanto,

$$O(n \log n + cn + m).$$

Como $c \leq k \leq n$ (Corolário 1), isso é $O(n^2+m)$ no pior caso, e $O(n \log n+m)$ quando o número de cores é limitado por uma constante. A memória usada é $O(n + m)$. O método de duas fases custa $O(n)$ na seleção e $O(k^2)$ no first-fit, que compara cada grupo com os anteriores, totalizando $O(n + k^2)$. O validador custa $O(k^2)$, e a leitura da instância, $O(n + m)$.

7.2. Limite para o número de cores

Teorema 1. *O Algoritmo 1 usa no máximo*

$$c \;\leq\; 1 + \max_{1 \leq i \leq k} \; \min_{v \in V_i} d_i(v)$$

cores, onde $d_i(v)$ é o número de vizinhos de v fora do grupo V_i.

Demonstração. Seja V_j o grupo que recebeu a última cor, c. Considere uma cor $\ell < c$. Durante a construção da cor ℓ, o grupo j esteve pendente o tempo todo, e todos os vértices de V_j foram examinados sem serem escolhidos. Pela condição da linha 8, cada $v \in V_j$ estava em B quando foi examinado, isto é, era vizinho de algum vértice u_ℓ já colocado na cor ℓ. Esse u_ℓ não pertence a V_j, pois nenhum vértice de V_j foi escolhido antes da cor c. Como cada vértice recebe no máximo uma cor, os vértices $u_1, \ldots, u_{c-1}$ são distintos. Logo, todo $v \in V_j$ tem pelo menos $c - 1$ vizinhos fora de V_j, ou seja, $c - 1 \leq \min_{v \in V_j} d_j(v) \leq \max_i \min_{v \in V_i} d_i(v)$. □

Corolário 1. *O algoritmo termina e usa no máximo* $\min\{k,\ \Delta + 1\}$ *cores, onde Δ é o grau máximo de G.*

Demonstração. No início de cada cor, $B = \emptyset$; o primeiro vértice de L cujo grupo está pendente é sempre escolhido. Assim, cada cor atende pelo menos um grupo, o laço termina e $c \leq k$. O limite $\Delta + 1$ segue do Teorema 1, pois $d_i(v) \leq \deg(v) \leq \Delta$. □

Como $\chi_P(G) \geq 1$, o Teorema 1 dá uma razão de aproximação de no máximo $1 + \max_i \min_{v \in V_i} d_i(v)$, que depende da instância e não é constante. Pela Seção 3, nenhuma heurística polinomial pode garantir razão $k^{1-\varepsilon}$, a menos que P = NP; por isso a qualidade do método é avaliada experimentalmente na próxima seção.

8. Experimentos

8.1. Metodologia

Os experimentos foram executados em um notebook com processador Intel Core i7-1365U, 32 GB de memória e Windows 11, usando CPython 3.13 em uma única *thread*. Os dois métodos são determinísticos e foram executados uma vez por instância. O tempo, medido com `time.perf_counter`, inclui apenas o algoritmo, sem a leitura do arquivo. Todas as soluções dos dois métodos, nas 280 instâncias, passaram pelo validador.

Os valores de referência foram retirados das Tabelas 2 e 3 de [Furini et al. 2018], que trazem os resultados de 175 instâncias distintas: 56 random, 32 nsfnet e 87 ring. Quando o limite superior de [Hoshino et al. 2011], reproduzido na Tabela 3, era melhor, ele foi usado. Com as cópias idênticas de `ring_n*p1.0`, esses valores cobrem 187 dos nossos arquivos. Em 180 deles o valor é ótimo comprovado; nos outros 7 (`n90p2t2s4`, `n90p3t2s4` e as cinco cópias de `ring_n20p1.0s1`) é o melhor valor conhecido. Para conferir que as instâncias são as mesmas, comparamos o tamanho dos grafos: n coincide em todos os casos, e o $|E|$ informado no artigo, que conta as arestas depois de transformar cada grupo em uma clique, coincide em 174 das 175 instâncias. A exceção, `n90p2t2s2`, tem o mesmo $|E|$ que `n90p2t2s4` no artigo, o que sugere um erro de digitação. O gap de um método é $100 \cdot (c - c^*)/c^*$, onde c^* é o valor de referência.

8.2. Resultados

A Tabela 2 resume os resultados por conjunto. A coluna "Melhor/igual/pior" conta as instâncias em que o guloso usou menos, o mesmo número ou mais cores que o método de duas fases. Os gaps são médias sobre as instâncias com valor de referência. A tabela completa, instância por instância, está no arquivo `resultados/tabela.md` do repositório.

Tabela 2. Resumo por conjunto: médias de cores e tempo sobre todas as instâncias; gaps sobre as instâncias com referência.

		Cores (média)		Melhor/	Tempo	Com	Gap médio (%)	
Conjunto	Inst.	Duas f.	Guloso	igual/pior	(ms)	ref.	Duas f.	Guloso
nsfnet	50	10,04	9,66	21/24/5	0,18	32	23,81	19,17
random	80	10,56	9,74	53/20/7	0,13	56	57,23	43,30
ring	150	22,63	22,70	49/40/61	0,67	99	18,84	18,83

A Tabela 3 detalha os resultados por grupo de instâncias, considerando apenas as instâncias com referência. A coluna $\Delta_{\max}$ é a maior diferença, em cores, entre o guloso e a referência dentro do grupo.

Para comparar com heurísticas construtivas publicadas, a Tabela 4 reúne as médias por conjunto random reportadas na Tabela 6.1 de [Fritz 2013] para Onestep-CD [Li and Simha 2000] e para a construção híbrida daquele trabalho, ao lado das médias dos nossos dois métodos nas mesmas instâncias. Os valores de [Fritz 2013] vêm de outra implementação, em outra máquina, por isso comparamos apenas o número de cores.

8.3. Discussão

Guloso contra duas fases. O guloso por classes usou, em média, 0,8 cor a menos que o método de duas fases nas instâncias random e 0,4 a menos nas nsfnet, sendo melhor ou igual em 91% e 90% dessas instâncias, respectivamente. Nas instâncias ring os dois métodos empatam na média (22,70 contra 22,63 cores, gap de 18,8% em ambos), e o guloso é pior em 61 das 150 instâncias. Medimos que, em ring e nsfnet, os dois métodos escolhem o mesmo representante em 97% e 96% dos grupos, contra 86% em random. Ou seja, nesses conjuntos a diferença está quase toda na coloração: a ordem dos grupos nos arquivos ring, que segue a enumeração das conexões da rede, parece favorecer o first-fit tanto quanto a ordem por grau favorece o guloso.

Tabela 3. Resultados por grupo, apenas nas instâncias com referência: médias de n, do valor de referência (c^*) e das cores de cada método, e gap médio.

					Cores (média)		Gap (%)		
Conjunto	Grupo	Inst.	n	c^*	Duas f.	Guloso	Duas f.	Guloso	$\Delta_{\max}$
random	$n = 60, p = 0{,}5$	1	60	5,00	9,00	9,00	80,0	80,0	4
	$n = 70, p = 0{,}5$	3	70	6,00	9,33	8,33	55,6	38,9	3
	$n = 80, p = 0{,}5$	5	80	6,00	10,40	9,20	73,3	53,3	4
	$n = 90, p = 0{,}5$	5	90	7,00	11,00	9,60	57,1	37,1	3
	$n = 100, p = 0{,}5$	5	100	7,00	11,80	10,40	68,6	48,6	4
	$n = 120, p = 0{,}5$	5	120	8,00	13,40	12,00	67,5	50,0	5
	$n = 90, p = 0{,}1$	5	90	2,80	4,20	3,60	53,3	30,0	1
	$n = 90, p = 0{,}2$	5	90	3,80	5,60	5,20	50,0	38,3	2
	$n = 90, p = 0{,}3$	5	90	5,00	7,40	7,00	48,0	40,0	2
	$n = 90, p = 0{,}4$	5	90	5,80	8,80	8,80	52,0	52,0	4
	$n = 90, p = 0{,}6$	4	90	8,25	13,25	11,75	61,1	42,7	4
	$n = 90, p = 0{,}8$	4	90	12,00	18,25	17,00	52,1	41,7	6
	$n = 90, p = 0{,}9$	4	90	16,00	22,25	21,75	39,1	35,9	7
nsfnet	$p = 0{,}4$	3	104	6,33	8,00	7,67	27,0	21,4	2
	$p = 0{,}5$	4	126	7,50	9,25	9,25	23,7	23,7	2
	$p = 0{,}6$	5	151	9,00	10,80	10,60	20,0	17,8	3
	$p = 0{,}7$	5	181	10,20	12,60	12,20	23,7	19,7	3
	$p = 0{,}8$	5	205	11,00	14,00	13,00	27,3	18,2	3
	$p = 0{,}9$	5	225	12,00	14,60	14,20	21,7	18,3	3
	$p = 1{,}0$	5	249	13,00	16,20	15,20	24,6	16,9	3
ring	10 nós	16	161	12,06	14,62	14,12	21,0	17,0	3
	15 nós	38	287	21,50	25,03	25,53	17,4	18,4	7
	20 nós	45	460	33,78	39,93	40,27	19,3	19,8	11

Distância até o ótimo. O guloso atingiu o valor de referência em apenas uma das 187 instâncias com referência (`n90p1t2s1`, com 3 cores), e o método de duas fases, em três. Em média, ele usa 1,9 cor a mais que a referência em nsfnet, 3,0 em random e 4,8 em ring. O gap relativo é maior em random (43%) porque essas instâncias têm poucos grupos e números cromáticos pequenos (de 2 a 16): uma única cor extra já representa de 6% a 50% do ótimo. Nas ring, o gap relativo é menor (19%), mas a diferença absoluta cresce com o tamanho da rede e chega a 11 cores nas maiores instâncias. Para comparação, a ALNS com busca local de [Furini et al. 2018], uma metaheurística que roda por muito mais tempo, encontra o ótimo em cerca de 158 de 171 instâncias e nunca fica mais de duas cores acima dele.

Comparação com heurísticas construtivas. Na Tabela 4, Onestep-CD usa de 0,2 a 1,4 cor a menos que o guloso em 14 dos 16 conjuntos, empata em $n = 120$ e perde em $n = 20$. A construção híbrida de [Fritz 2013] é melhor ainda. A diferença é esperada: Onestep-CD recalcula, a cada passo, quantas cores diferentes cercam cada vértice (o grau de cor), enquanto o nosso guloso usa uma ordem fixa, calculada uma única vez a partir do grau. Essa ordem fixa é o que mantém o método simples e muito rápido.

Tabela 4. Média de cores por conjunto random (cinco instâncias cada): métodos deste trabalho e heurísticas construtivas reportadas por [Fritz 2013].

n	p	Duas fases	Guloso	Onestep-CD	Híbrida
20	0,5	3,8	3,4	3,6	3,4
40	0,5	5,8	6,0	5,6	5,0
60	0,5	8,2	7,6	7,4	6,6
70	0,5	9,2	8,4	7,6	7,0
80	0,5	10,4	9,2	8,6	8,0
90	0,5	11,0	9,6	9,4	9,4
100	0,5	11,8	10,4	10,2	10,0
120	0,5	13,4	12,0	12,0	11,0
90	0,1	4,2	3,6	3,0	3,0
90	0,2	5,6	5,2	5,0	4,2
90	0,3	7,4	7,0	6,2	5,8
90	0,4	8,8	8,8	7,8	7,6
90	0,6	13,2	11,8	11,2	10,4
90	0,7	15,2	14,0	13,6	12,6
90	0,8	18,6	17,4	16,4	15,6
90	0,9	22,4	21,4	20,0	19,8

Tempo. O guloso levou no máximo 3,4 ms por instância (nas maiores, as cópias de `ring_n20p1.0s1`, com 760 vértices e 117 mil arestas) e cerca de 0,12 s para as 280 instâncias. O crescimento acompanha o termo $cn + m$ da análise: nas ring, m domina e o tempo cresce com o número de arestas. Os métodos exatos de [Furini et al. 2018] levam de segundos a uma hora nas mesmas instâncias, o que torna o guloso adequado para gerar soluções iniciais.

9. Conclusão

Este trabalho estudou o Problema de Coloração Seletiva de Grafos, NP-difícil e sem aproximação com razão constante, e implementou uma heurística gulosa que monta uma classe de cor por vez. Mostramos que ela roda em $O(n \log n + cn + m)$ e usa no máximo $1 + \max_i \min_{v \in V_i} d_i(v)$ cores. Nas 280 instâncias da literatura, todas as soluções foram válidas, o tempo ficou abaixo de 4 ms, e o método superou um método de duas fases nas instâncias random e nsfnet, empatando nas ring. A distância até os valores ótimos (19% em nsfnet e ring, 43% em random) e a comparação com Onestep-CD mostram que a ordem fixa por grau limita a qualidade das soluções.

Como trabalhos futuros, sugerimos: (i) trocar a ordem fixa por uma ordem dinâmica, como o grau de cor de Onestep-CD; (ii) aplicar uma busca local que esvazia a menor classe de cor e tenta reinserir os seus grupos nas demais, como em [Furini et al. 2018]; e (iii) executar várias vezes com desempates aleatórios, ficando com a melhor solução.

Referências

Brélaz, D. (1979). New methods to color the vertices of a graph. *Communications of the ACM*, 22(4):251–256.

Demange, M., Ekim, T., Ries, B., and Tanasescu, C. (2015). On some applications of the selective graph coloring problem. *European Journal of Operational Research*, 240(2):307–314.

Demange, M., Monnot, J., Pop, P., and Ries, B. (2014). On the complexity of the selective graph coloring problem in some special classes of graphs. *Theoretical Computer Science*, 540:89–102.

Fritz, G. (2013). A hybrid algorithm for the partition coloring problem. Diplomarbeit, Technische Universität Wien.

Frota, Y., Maculan, N., Noronha, T. F., and Ribeiro, C. C. (2010). A branch-and-cut algorithm for partition coloring. *Networks*, 55(3):194–204.

Furini, F., Malaguti, E., and Santini, A. (2018). An exact algorithm for the partition coloring problem. *Computers & Operations Research*, 92:170–181.

Garey, M. R. and Johnson, D. S. (1979). *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, San Francisco.

Hoshino, E. A., Frota, Y. A., and de Souza, C. C. (2011). A branch-and-price approach for the partition coloring problem. *Operations Research Letters*, 39(2):132–137.

Karp, R. M. (1972). Reducibility among combinatorial problems. In Miller, R. E. and Thatcher, J. W., editors, *Complexity of Computer Computations*, pages 85–103. Plenum Press, New York.

Li, G. and Simha, R. (2000). The partition coloring problem and its application to wavelength routing and assignment. In *Proceedings of the First Workshop on Optical Networks*, Dallas, TX.

Noronha, T. F. and Ribeiro, C. C. (2006). Routing and wavelength assignment by partition colouring. *European Journal of Operational Research*, 171(3):797–810.

Pop, P. C., Hu, B., and Raidl, G. R. (2013). A memetic algorithm for the partition graph coloring problem. In *Extended Abstracts of the 14th International Conference on Computer Aided Systems Theory (EUROCAST)*, pages 167–169, Gran Canaria, Espanha.

Santini, A. (2017). alberto-santini/selective-graph-colouring. `https://github.com/alberto-santini/selective-graph-colouring`. Zenodo, doi:10.5281/zenodo.806149. Acesso em out. 2026.

Şeker, O., Ekim, T., and Taşkın, Z. C. (2021). An exact cutting plane algorithm to solve the selective graph coloring problem in perfect graphs. *European Journal of Operational Research*, 291(1):67–83.

Welsh, D. J. A. and Powell, M. B. (1967). An upper bound for the chromatic number of a graph and its application to timetabling problems. *The Computer Journal*, 10(1):85–86.

Zuckerman, D. (2007). Linear degree extractors and the inapproximability of max clique and chromatic number. *Theory of Computing*, 3:103–128.